From idea to prototype

A concise summary of the EcoFlow Campus concept, development approach, tools, implemented features, and current evaluation.

THE INITIAL IDEA

Build a demo-ready, evidence-led resource dashboard for an eight-building campus. Combine electricity, water, occupancy, waste, weather, and operational records to make campus resource information easier to review and act on.

The concept emphasizes data quality, waste classification, short-horizon forecasting, offline resilience, explainability, and honest reporting.

Development process

> **1. Inspect the supplied data**

Reviewed the seven workbook domains: buildings, calendar/weather, daily summaries, hourly readings, manual logs, and labelled/unlabelled waste events.

> **2. Build and refine the prototype**

Implemented the dashboard and API, tested existing data/forecast helpers, then expanded building history into campus-wide dataset and date/time filters.

> **3. Verify and communicate accurately**

Added focused tests, checked frontend type/build, measured the current rule classifier on a labelled holdout, and documented limitations for the demo.

Tools used

AI assistance: GitHub Copilot in VS Code via Copilot SDK; the underlying model/version was not recorded.
Engineering stack: React, TypeScript, Vite, Recharts, Python, FastAPI, pandas, NumPy, scikit-learn, SQLite helpers, and pytest.

Human contribution: selected the campus sustainability problem, supplied the datasets and requirements, chose the EcoFlow Campus name, directed feature priorities, and requested historical filtering and visible branding. AI-assisted changes were integrated and validated in the repository.

Features and final outcome

What the prototype demonstrates

- Campus overview with building cards and latest available electricity, water, and footfall readings.
- Building detail with resource selection, a 6/12-hour baseline forecast panel, and observed hourly history chart/table with resource and date/time filters.
- Campus-wide History page with dataset, metric, building, and date/time filters for hourly streams, daily summaries, calendar/weather, labelled and unlabelled waste events, and manual logs.
- Waste classifier screen showing rule-based predictions from unlabelled test events; it does not report accuracy for that unlabelled file.
- Data audit artifacts, ingestion/normalization helpers, edge-buffer and reconciliation helpers, and a documented demo flow.

Measured classifier result

88.2%

accuracy on a stratified 20% holdout (1,263 records)

Macro F1 76.3% | balanced accuracy 76.4%

E-waste recall only 10.0% (7 of 70 holdout cases)

This evaluates the existing sensor-threshold rules on labelled benchmark training data; it is not an independent real-world test or an ML-model result. The separate test dataset has no labels.

Outcome and limitations

- Validation at this snapshot: 66 Python tests passed, 2 skipped; frontend typecheck and production build passed.
- The forecast endpoint uses a recent-history baseline rather than a trained forecasting model. Waste classification in the app is heuristic, not the trained model pipeline.
- The supplied data is synthetic/hypothetical benchmark data, not live campus telemetry. Treat readings and results as prototype demonstrations.
- Anomaly, incentive, connectivity, data-quality, and cost/scale capabilities are not all fully connected to the interface; do not present placeholders as measured outcomes.

Final outcome: a working, locally runnable campus-data prototype with interactive history and transparent evaluation, plus a clear roadmap for model integration and operational validation.